МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Наследование

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ с использованием механизма наследования.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.
2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы (при нехватке заданий вариант задания вычисляется как *номер_в_списке_группы - количество_заданий*).
3. Разработать классы согласно варианту задания.
4. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:
   - титульный лист;
   - цель лабораторной работы;
   - постановку задачи;
   - текст программы с комментариями;
   - демонстрацию работы программы *(Снимки экрана при выполнении действий программы с описанием).*
   - краткие ответы на контрольные вопросы;
   - выводы по лабораторной работе.
5. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу на языке Python, реализующую задачу ранее решенную на языке C++ в лабораторной работе №2.

Необходимо разработать программу, в которой будет реализовано несколько классов согласно варианту задания.

В реализуемой программе необходимо предусмотреть возможность демонстрации работы методов созданных классов.

Организацию взаимодействия с пользователем реализовать **на основе меню**.

В зависимости от оценки, на которую вы претендуете, необходимо выполнить следующие задания (для каждой следующей оценки нужно выполнить **ВСЕ** предыдущие задания, если обратное не сказано явно):

Оценка	Общее задание
Удовлетворительно	1. Определить базовый класс (обозначим его Ab) с обычными и абстрактными методами (согласно варианту задания) и методом my_name(), который возвращает строку с названием класса. Все поля должны быть защищёнными (protected). Все методы работают только с одним или двумя экземплярами (не списком).
	2. Определить классы-наследники класса Ab (согласно варианту задания). Реализовать в них перегрузку абстрактных методов класса Ab. Также все поля должны быть закрытыми (private), все методы работают только с одним или двумя экземплярами (не списком).
	3. В основной программе создать список и заполнить его экземплярами всех классов-наследников. Реализовать вывод одного из списков, предусмотренных вариантом.
Хорошо	4. Реализовать вывод остальных списков, предусмотренных вариантом.
	5. Заполнять список экземпляров (задание 3) из текстового файла, выводить данные также в файл.
Отлично	6. Реализовать класс (обозначим его Ms), полями которого будет список экземпляров классов-наследников класса Ab. Вывод списков из заданий 3 и 4 сделать отдельными методами класса Ms (в этом случае можно писать их сразу в классе Ms без отдельной реализации в основной программе).
	7. В основной программе создать экземпляр класса Ms и работать с ним.

Вариант №5.

Создать класс **MediaStorage** (Портативный накопитель), включающий базовые характеристики.

На его основе реализовать классы **PortableHDD** (Портативный жесткий диск) и **DVD** (DVD накопитель).

Наряду с общими полями (название, емкость, фирма и цена) каждый класс должен содержать и специфичные для него поля.

Для массива указателей на объекты этих классов предусмотреть возможность:

- вывода списка накопителей, у которых емкость лежит в заданном диапазоне;
- вывода статистической сводки (какие минимальные/максимальные значения соотношения объём/цена);
- вывода массива, отсортированного по возрастанию цены.

Создать класс **Shop**, содержащий массив указателей на объекты этих классов.

Написать демонстрационную программу, в которой будут использоваться все методы классов.

ХОД РАБОТЫ

Файл **header**.py:

```python
import os
from abc import ABC, abstractmethod

class MediaStorage(ABC):
    """
    Абстрактный класс, представляющий собой Накопитель памяти.
    Поля:
    - `name (str)`: Название.
    - `capacity (int)`: Ёмкость.
    - `manufacturer (str)`: Производитель.
    - `price (float)`: Цена.
    """
    def __init__(self, name="", capacity=0.0, manufacturer="", price=0.0):
        self._name = name.strip()
        self._capacity = float(capacity)
        self._manufacturer = manufacturer.strip()
        self._price = float(price)

    # Геттеры для общих полей
    def get_name(self): return self._name
    def get_capacity(self): return self._capacity
    def get_manufacturer(self): return self._manufacturer
    def get_price(self): return self._price

    def my_name(self):
        return type(self).__name__

    # Абстрактный метод для наследования.
    # Возвращает строку с данными полей объекта.
    # Используется для записи в файл.
    @abstractmethod
    def stats_to_line(self):
        pass

    # Возвращает строку с данными полей объекта.
    # Используется для вывода в консоль.
    # Переопределяется в дочерних классах.
    def data_output(self):
        return (f"Тип: {self.my_name()},\n"
                f"Название: {self._name},\n"
                f"Ёмкость: {self._capacity} ГБ,\n"
                f"Производитель: {self._manufacturer},\n"
                f"Цена: {self._price:.2f}, руб.")

class PortableHDD(MediaStorage):
    """
    Класс, представляющий Жёсткий диск.
    Дополнительные поля:
    - `rpm (int)`: RPM (обороты в минуту).
    """
    def __init__(self, name="", capacity=0.0, manufacturer="", price=0.0, rpm=0):
        super().__init__(name, capacity, manufacturer, price)
        self.__rpm = int(rpm)

    def get_rpm(self): return self.__rpm

    def data_output(self):
        return (f"{super().data_output()},\n"
                f"RPM: {self.__rpm}.")

```

```python
    def stats_to_line(self):
        return f"{self._name},{self._capacity},{self._manufacturer},{self._price:.2f},{self.__rpm}"


class DVD(MediaStorage):
    """
    Класс, представляющий DVD накопитель.
    Дополнительные поля:
    - `type_dvd (str)`: Тип DVD (например, DVD-R, DVD+R, DVD-RW).
    """
    def __init__(self, name="", capacity=0.0, manufacturer="", price=0.0, type_dvd="DVD-R"):
        super().__init__(name, capacity, manufacturer, price)
        self.__type_dvd = type_dvd.strip()

    def get_type_dvd(self): return self.__type_dvd

    def data_output(self):
        return (f"{super().data_output()},\n"
                f"Тип: {self.__type_dvd}.")

    def stats_to_line(self):
        return f"{self._name},{self._capacity},{self._manufacturer},{self._price:.2f},{self.__type_dvd}"

class Shop:
    """
    Класс-контейнер для списка накопителей (экземпляров классов, наследующихся от MediaStorage).
    """
    def __init__(self):
        self.__storage_devices = []

    def add_device(self, device):
        self.__storage_devices.append(device)

    def load_from_file(self, filename):
        if not os.path.exists(filename):
            print(f"Файл {filename} не найден.")
            return

        with open(filename, 'r', encoding='utf-8') as f:
            lines = f.readlines()

        self.__storage_devices = [] # Предварительно очистим список
        for line in lines:
            parts = line.strip().split(',')
            device_type, name, capacity, manufacturer, price, extra = parts
            match device_type:
                case "PortableHDD":
                    self.__storage_devices.append(PortableHDD(name, capacity, manufacturer, price, extra))
                case "DVD":
                    self.__storage_devices.append(DVD(name, capacity, manufacturer, price, extra))
                case _:
                    print(f"Неизвестный тип устройства: {device_type}. Пропускаю строку: {line}")
        print(f"Данные загружены из {filename}.")

    def save_to_file(self, filename):
        if not self.__storage_devices:
            print("Список накопителей пуст.")
            return
        with open(filename, 'w', encoding='utf-8') as f:
```

```python
        with open(filename, 'w', encoding='utf-8') as f:
            for device in self.__storage_devices:
                f.write(device.my_name() + ',' + device.stats_to_line() + "\n")
        print(f"Данные сохранены в {filename}")

    def display_devices(self):
        if not self.__storage_devices:
            print("Список накопителей пуст.")
            return
        for i, device in enumerate(self.__storage_devices):
            print(f"--- Накопитель {i+1} ---")
            print(device.data_output())

    # Выборка по ёмкости в диапазоне
    def select_by_capacity_range(self, min_cap, max_cap):
        return [d for d in self.__storage_devices if min_cap <= d.get_capacity() <= max_cap]

    # Вывод статистической сводки
    def get_stats_by_type(self):
        if not self.__storage_devices:
            print("Список накопителей пуст.")
            return
        stats = {}
        # Собираем статистику
        for device in self.__storage_devices:
            name = device.my_name()
            if name not in stats:
                stats[name] = {'count': 0, 'total_price': 0.0}
            stats[name]['count'] += 1
            stats[name]['total_price'] += device.get_price()

        result_str = "\n--- Статистическая сводка ---\n"
        for device_type, data in stats.items():
            avg_price = data['total_price'] / data['count']
            print("----------")
            result_str += f"Тип: {device_type}\n"
            result_str += f"Количество: {data['count']}\n"
            result_str += f"Общая цена: {data['total_price']:.2f}\n"
            result_str += f"Средняя цена: {avg_price:.2f}\n"
        return result_str

    # Сортировка массива по возрастанию цены
    def sort_by_price(self):
        self.__storage_devices.sort(key=lambda d: d.get_price())
```

Файл **main.py**:

```python
from header import Shop

# Инициализируем список накопителей
container = Shop()
# Файл для записи/чтения
filename = "storage_devices.txt"

# Заполняем список начальными данными
with open(filename, "w", encoding='utf-8') as f:
    f.write("PortableHDD,WD Blue,1000,Western Digital,5000.0,5400\n")
    f.write("DVD,Disk A,4.7,Generic,100.0,DVD-R\n")
    f.write("PortableHDD,Seagate Fast,2000,Seagate,8000.0,7200\n")
    f.write("DVD,Disk B,4.7,Verbatim,150.0,DVD+R\n")
    f.write("PortableHDD,Toshiba Canvio,500,Toshiba,3500.0,5400\n")
    f.write("DVD,Disk C,9.4,Verbatim,200.0,DVD-RW\n")

while True:
    print("\n--- База данных магазина накопителей ---")
    print("1. Загрузить накопители из файла")
    print("2. Сохранить накопители в файл")
    print("3. Показать все накопители")
    print("4. Выбрать накопители по ёмкости в диапазоне")
    print("5. Показать статистику по типам накопителей")
    print("6. Отсортировать список накопителей по цене")
    print("0. Выйти")
    choice = input("Выберите действие: ")
    match choice:
        case '1':
            container.load_from_file(filename)
        case '2':
            container.save_to_file(filename)
        case '3':
            container.display_devices()
        case '4':
            min_c = int(input("Введите минимальную ёмкость: "))
            max_c = int(input("Введите максимальную ёмкость: "))
            selected = container.select_by_capacity_range(min_c, max_c)
            print(f"\n--- Накопители с ёмкостью от {min_c} до {max_c} Гб ---")
            for d in selected:
                print("----------")
                print(d.data_output())
        case '5':
            stats = container.get_stats_by_type()
            if(not stats):
                continue
            print(stats)
        case '6':
            container.sort_by_price()
            print("Список накопителей отсортирован по цене.")
        case '0':
            print("Выход из программы...")
            break
        case _:
            print("Неверный выбор. Пожалуйста, попробуйте снова.")
```

Загрузка данный из файла:

```
--- База данных магазина накопителей ---
1. Загрузить накопители из файла
2. Сохранить накопители в файл
3. Показать все накопители
4. Выбрать накопители по ёмкости в диапазоне
5. Показать статистику по типам накопителей
6. Отсортировать список накопителей по цене
0. Выйти
Выберите действие: 1
Данные загружены из storage_devices.txt.
```

storage_devices.txt

C_codespace > 2 kurs > 3-rd senester > Programming languages > laboratory work

```
PortableHDD,WD Blue,1000,Western Digital,5000.0,5400
DVD,Disk A,4.7,Generic,100.0,DVD-R
PortableHDD,Seagate Fast,2000,Seagate,8000.0,7200
DVD,Disk B,4.7,Verbatim,150.0,DVD+R
PortableHDD,Toshiba Canvio,500,Toshiba,3500.0,5400
DVD,Disk C,9.4,Verbatim,200.0,DVD-RW
```

Вывод данных в консоль:

```
3. Показать все накопители
4. Выбрать накопители по ёмкости в диапазоне
5. Показать статистику по типам накопителей
6. Отсортировать список накопителей по цене
0. Выйти
Выберите действие: 3
--- Накопитель 1 ---
Тип: PortableHDD,
Название: WD Blue,
Ёмкость: 1000.0 ГБ,
Производитель: Western Digital,
Цена: 5000.00, руб.,
RPM: 5400.
--- Накопитель 2 ---
Тип: DVD,
Название: Disk A,
Ёмкость: 4.7 ГБ,
Производитель: Generic,
Цена: 100.00, руб.,
Тип: DVD-R.
--- Накопитель 3 ---
Тип: PortableHDD,
Название: Seagate Fast,
Ёмкость: 2000.0 ГБ,
Производитель: Seagate,
Цена: 8000.00, руб.,
RPM: 7200.
--- Накопитель 4 ---
Тип: DVD,
Название: Disk B,
Ёмкость: 4.7 ГБ,
Производитель: Verbatim,
Цена: 150.00, руб.,
Тип: DVD+R.
--- Накопитель 5 ---
Тип: PortableHDD,
Название: Toshiba Canvio,
Ёмкость: 500.0 ГБ,
Производитель: Toshiba,
Цена: 3500.00, руб.,
RPM: 5400.
--- Накопитель 6 ---
Тип: DVD,
Название: Disk C,
Ёмкость: 9.4 ГБ,
Производитель: Verbatim,
Цена: 200.00, руб.,
Тип: DVD-RW.
```

Выборка накопителей по ёмкости:

```
4. Выбрать накопители по ёмкости в диапазоне
5. Показать статистику по типам накопителей
6. Отсортировать список накопителей по цене
0. Выйти
Выберите действие: 4
Введите минимальную ёмкость: 500
Введите максимальную ёмкость: 1500

--- Накопители с ёмкостью от 500 до 1500 Гб ---
----------
Тип: PortableHDD,
Название: WD Blue,
Ёмкость: 1000.0 ГБ,
Производитель: Western Digital,
Цена: 5000.00, руб.,
RPM: 5400.
----------
Тип: PortableHDD,
Название: Toshiba Canvio,
Ёмкость: 500.0 ГБ,
Производитель: Toshiba,
Цена: 3500.00, руб.,
RPM: 5400.
```

Статистическая сводка по типам накопителей:

```
5. Показать статистику по типам накопителей
6. Отсортировать список накопителей по цене
0. Выйти
Выберите действие: 5
----------
----------

--- Статистическая сводка ---
Тип: PortableHDD
Количество: 3
Общая цена: 16500.00
Средняя цена: 5500.00
Тип: DVD
Количество: 3
Общая цена: 450.00
Средняя цена: 150.00
```

Сортировка списка накопителей:

```
6. Отсортировать список накопителей по цене
0. Выйти
Выберите действие: 6
Список накопителей отсортирован по цене.
```

```
Выберите действие: 3
--- Накопитель 1 ---
Тип: DVD,
Название: Disk A,
Ёмкость: 4.7 ГБ,
Производитель: Generic,
Цена: 100.00, руб.,
Тип: DVD-R.
--- Накопитель 2 ---
Тип: DVD,
Название: Disk B,
Ёмкость: 4.7 ГБ,
Производитель: Verbatim,
Цена: 150.00, руб.,
Тип: DVD+R.
--- Накопитель 3 ---
Тип: DVD,
Название: Disk C,
Ёмкость: 9.4 ГБ,
Производитель: Verbatim,
Цена: 200.00, руб.,
Тип: DVD-RW.
--- Накопитель 4 ---
Тип: PortableHDD,
Название: Toshiba Canvio,
Ёмкость: 500.0 ГБ,
Производитель: Toshiba,
Цена: 3500.00, руб.,
RPM: 5400.
--- Накопитель 5 ---
Тип: PortableHDD,
Название: WD Blue,
Ёмкость: 1000.0 ГБ,
Производитель: Western Digital,
Цена: 5000.00, руб.,
RPM: 5400.
--- Накопитель 6 ---
Тип: PortableHDD,
Название: Seagate Fast,
Ёмкость: 2000.0 ГБ,
Производитель: Seagate,
Цена: 8000.00, руб.,
RPM: 7200.
```

Сохранение данных в файл:

```
2. Сохранить накопители в файл
3. Показать все накопители
4. Выбрать накопители по ёмкости в диапазоне
5. Показать статистику по типам накопителей
6. Отсортировать список накопителей по цене
0. Выйти
Выберите действие: 2
Данные сохранены в storage_devices.txt
```

storage_devices.txt

```
1   DVD,Disk A,4.7,Generic,100.00,DVD-R
2   DVD,Disk B,4.7,Verbatim,150.00,DVD+R
3   DVD,Disk C,9.4,Verbatim,200.00,DVD-RW
4   PortableHDD,Toshiba Canvio,500.0,Toshiba,3500.00,5400
5   PortableHDD,WD Blue,1000.0,Western Digital,5000.00,5400
6   PortableHDD,Seagate Fast,2000.0,Seagate,8000.00,7200
7
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **В чем состоит сущность механизма наследования как принципа ООП?**
   Это механизм, позволяющий одному классу (производному) получать свойства и методы другого класса (базового). Это обеспечивает повторное использование кода, упрощает разработку и поддержку, а также позволяет создавать иерархии объектов.
2. **Как объявляется производный класс?**
   class children_class(parent_class)
3. **Что такое экземпляр класса и как его создать в программе?**
   Через **self.var_name**. Договорённости: **_var** — защищённое поле, не предназначенное для прямого доступа; **__var** — приватное поле, для внутреннего использования только в классе.
4. **Как происходит перегрузка методов при наследовании?**
   Перегрузка — это переопределение метода базового класса в производном классе, т.е. метод с тем же именем в производном классе заменяет метод базового, что переопределяет его поведении при вызове данного метода у объекта производного класса.
5. **Каким образом выполняется инициализация наследуемых элементов?**
   В конструкторе производного класса с помощью конструкции **super().__init__(…)** или явного вызова конструктора базового класса **base_class.__init__(self, …)**
6. **В каком порядке вызываются конструкторы класса при создании экземпляра производного класса?**
   Сначала вызывается конструктор базового класса, затем производного.
7. **Что такое абстрактный класс?**
   Это класс, который не может быть создан напрямую, и который служит шаблоном для производных классов. Он содержит абстрактные методы (объявленные, но без реализации), которые предназначены для переопределения в дочерних классах.
8. **Что такое множественное наследование?**
   Это возможность создания класса, который наследует атрибуты сразу от нескольких базовых классов.
   Синтаксис: **class children_class(parent_class_A, parent_class_B)**

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ на языке Python, использующих механизм наследования.